Proposed Atomic Interest-Rate Refresh Plan

Status	Proposal for community and external technical review. No mainnet execution has been approved.
Network	Hedera Mainnet, chain ID 295.

Purpose

Bonzo Finance has deployed and wired new interest-rate strategies for WHBAR, USDC, and WETH. The LendingPool remains paused and all reserves remain frozen.

Wiring changes the strategy used for future rate calculations, but it does not refresh the stored variable borrow and liquidity rates. That requires a LendingPool interaction that calls `updateState()` and `updateInterestRates()`.

The current implementation exposes no admin function that performs this refresh while paused. Every available entry point that reaches `updateInterestRates()` is gated by `whenNotPaused`, so a temporary unpause is unavoidable unless the protocol implementation is changed.

This proposal describes how to perform the refresh without creating a publicly accessible unpause window between separate transactions.

Why the existing refresh flow is unsafe

The current script uses five transactions:

```
unpause -> deposit WHBAR -> deposit USDC -> deposit WETH -> pause
```

Once the first transaction finalises, the pool remains publicly accessible until the final pause transaction completes. Disabling the Bonzo UI does not prevent direct contract calls.

During that window:

- liquidation bots already targeting the paused pool can execute;
- withdrawal bots can compete for limited reserve liquidity on a first-come-first-served basis;
- flash loans and aToken transfers become available;
- external transactions can be placed between Bonzo's transactions; and
- variable borrowing becomes available on any target reserve that is unfrozen while borrowing remains enabled.

Freezing does not block withdrawals or liquidations. Parallel submission does not remove the window because cross-signer ordering is not guaranteed, same-signer transactions require nonce sequencing, and external transactions can still be interleaved.

Proposed mechanism

Bonzo Finance proposes a non-upgradeable, one-shot `AtomicRatePokeExecutor.sol` that completes the refresh inside one EVM transaction.

The executor would temporarily receive only the emergency-admin role. This permits it to call `setPoolPause(bool)` but does not grant authority over strategies, reserve configuration, borrowing settings, pool administration, or protocol ownership. The role is needed only for pause control; `flashLoan()` is permissionless once the pool is unpaused.

The AddressesProvider owner and pool admin remain unchanged. The owner can replace the emergency admin at any time without approval from the executor. Pool-admin authority alone cannot do this.

Atomic operation

```
verify pool is paused
verify executor is emergency admin
verify approved WHBAR, USDC, and WETH strategies are wired

unpause LendingPool

flashLoan:
  assets  = [WHBAR, USDC, WETH]
  amounts = [one atomic unit, one atomic unit, one atomic unit]
  modes   = [0, 0, 0]

executeOperation:
  require msg.sender == LendingPool
  require initiator == executor
  approve exact amount + premium for each asset
  return true

pause LendingPool
require pool is paused
mark executor used
```

For every asset in a mode-`0` flash loan, the LendingPool calls `updateState()` and `updateInterestRates()`. The flash-loan path does not reject frozen reserves, so all three rates can be refreshed without unfreezing them.

The configured premium is 9 basis points and is calculated using integer division: `amount * 9 / 10000`. A one-unit loan produces a zero premium and repays exactly one unit. Any larger amount requires the executor to hold the additional premium.

External transactions cannot interleave inside one EVM transaction. Callers observe the pool as paused before and after it. If any internal call fails, the entire transaction reverts, including the unpause, leaving the pool paused and the stored rates unchanged.

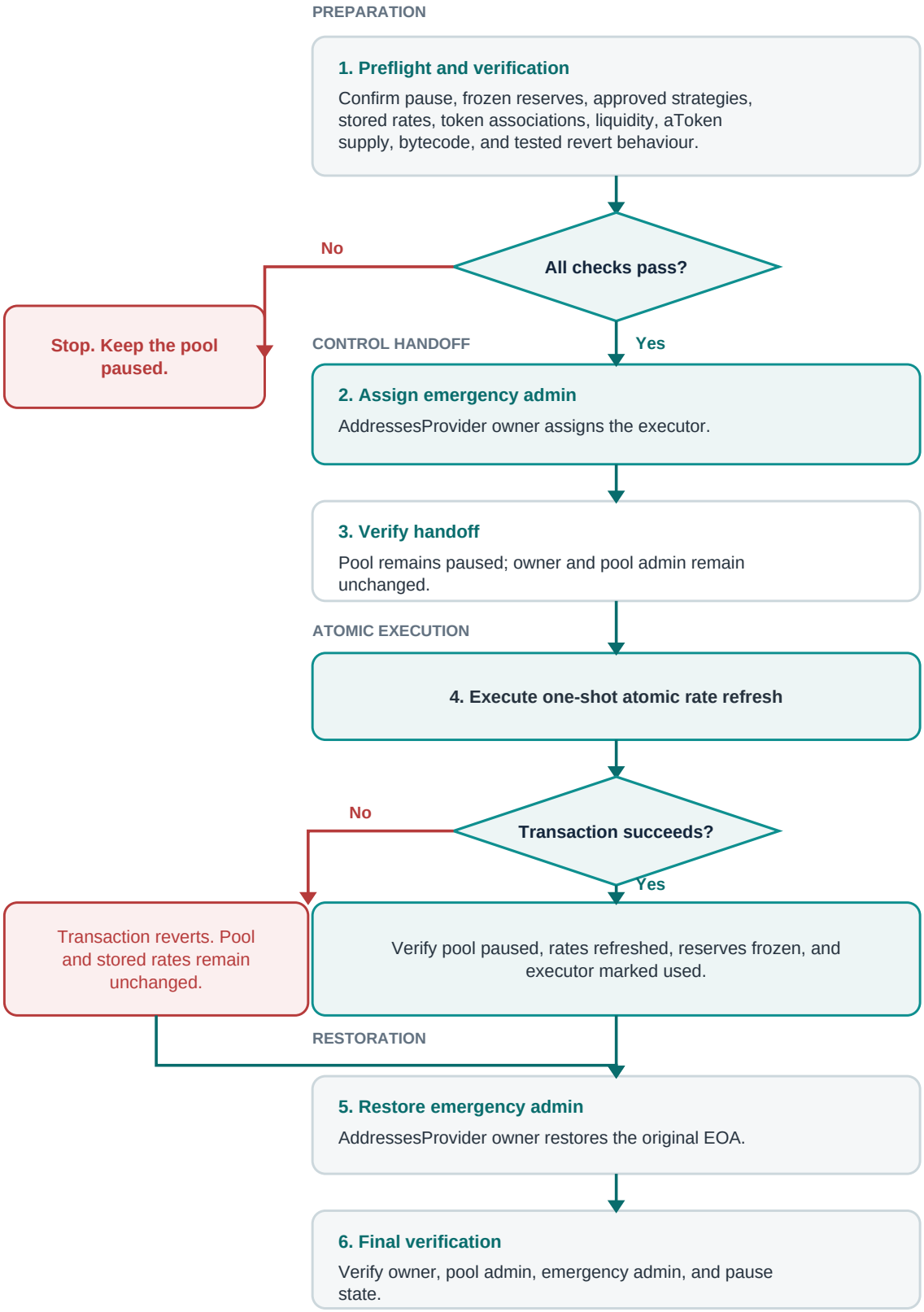
Executor requirements

The executor must:

- be non-upgradeable, one-shot, source-verified, and restricted to the current Bonzo admin EOA;
- use immutable AddressesProvider, configurator, LendingPool, strategy, and asset addresses;
- expose no arbitrary call, `delegatecall`, generic batch, or standalone unpause function;
- never receive the pool-admin role or AddressesProvider ownership;
- associate with each HTS asset it must receive and prove it can receive and return all three assets;
- require `msg.sender == LendingPool` and `initiator == address(this)` in `executeOperation()`;
- approve only the exact `amount + premium` required for repayment;
- preflight that each aToken holds the requested underlying amount and has non-zero total supply;
- allow a failed final pause to revert the entire transaction without `try/catch`; and
- expose a controller-only rescue function that can pause but never unpause.

`flashLoan()` does not use a `nonReentrant` modifier, making callback-caller and initiator validation mandatory.

Execution and recovery flow



The executor does not return the role itself. If it is compromised or unusable while the pool is paused, the AddressesProvider owner restores the original EOA as emergency admin and abandons the executor.

If the pool is unexpectedly open, the controller first calls the executor's pause-only rescue function. If that fails, recovery requires two transactions:

1. The AddressesProvider owner restores the original EOA as emergency admin.
2. The restored EOA pauses the pool.

The protocol remains exposed between these two fallback transactions.

Risks requiring external review

- A missing access check could permit an unauthorised unpause or execution.
- Incorrect immutable addresses, HTS associations, available liquidity, callback validation, premium calculation, approval, or repayment could make execution revert.
- Catching or suppressing a failed final pause could leave the pool open and must be prohibited.
- While the executor holds the role, the EOA cannot pause directly without first restoring itself as emergency admin.
- A compromised controller could invoke any unpause path exposed by the executor.
- Restoring the wrong address or leaving the executor as emergency admin would preserve unnecessary authority or delay emergency control.
- Recovery depends on the AddressesProvider owner remaining available and uncompromised.

Feedback requested

Bonzo Finance is seeking review of:

1. Whether a temporary, narrowly scoped emergency-admin executor is an acceptable control model.
2. Whether a multi-asset flash loan is the safest way to refresh all three reserves while keeping them frozen.
3. Whether the atomicity and revert assumptions hold across the Bonzo contracts and Hedera EVM execution.
4. Whether additional access-control, callback, recovery, or post-execution checks are required.
5. Whether any existing deployed-code path can safely refresh the stored rates without a temporary unpause. Bonzo Finance has not found one in the current implementation.